Innopolis University

Big Data — Final Project, 2026

Detecting IoT Malware at Network Scale

An end-to-end Hadoop / Spark pipeline over the Aposemat IoT-23 corpus

Team: Team 30
Repository: Dmitry057/big-data-group-project
Local path: `/home/team30/project30`

Ivan Ilyichev	Data Engineer
Kirill Shumsky	ML Specialist
Nikita Zagainov	Data Scientist
Dmitry Tetkin	Tester & Technical Writer

May 10, 2026

1 Introduction

1.1 Why we picked this problem

Internet-of-Things devices — cheap routers, IP cameras, DVRs — ship with default passwords and almost never get patched. Once compromised, they are pulled into botnets like *Mirai*, *Hide-and-Seek* and *Torii*, and used to generate DDoS attacks, port scans and command-and-control traffic. The malicious flows look almost identical to the device's own normal traffic, and there are millions of them per hour on a real network. So we wanted a system that can sit in front of that firehose, look at every connection, and say: *this one is fine, this one is part of a port scan, this one is a DDoS amplifier.*

The IoT-23 dataset gave us labelled, real-world data to do exactly that — roughly 25 million labelled connections from 12 IoT-device captures. Concretely, the goals of the project are:

1. Move 25M Zeek connection records from CSV into a Hive warehouse on HDFS using the standard Hadoop tools.
2. Run a small set of HiveQL queries to understand what's actually in the data — how the classes split, which ports get attacked, what the typical packet shapes look like.
3. Train a distributed classifier in Spark that tells the seven classes apart, evaluate it on a held-out set, and put everything in a Superset dashboard the rest of the team can read.

The whole thing is wired into a single `main.sh` so the grader can re-run every stage from scratch.

1.2 What's in this document

- **Data Description** — where the data comes from, how big it is, and how the four pipeline stages connect.
- **Data Preparation** — the relational schema, how we loaded it into Postgres and lifted it into Hive, and the storage-format benchmark.
- **Data Analysis** — seven HiveQL insights, with the chart and a short interpretation for each.
- **ML Modeling** — the Spark feature pipeline, the two models we trained, and how they did on the test set.
- **Data Presentation** — what's on the Superset dashboard and what it tells the viewer.
- **Conclusion** — what we shipped, what gave us trouble, what we'd do differently, and the contribution table.

1.3 Pipeline at a glance

Stage	Script	Output
1 — Ingestion	`stage1.sh`	PostgreSQL DB + HDFS Parquet warehouse
2 — Storage & EDA	`stage2.sh`	Hive partitioned tables + 7 EDA result tables
3 — Predictive analytics	`stage3.sh`	Trained Spark ML models + held-out predictions
4 — Dashboard	`stage4.sh`	Apache Superset dashboard refreshed

Table 1: The four stages of the pipeline.

2 Data Description

2.1 Where the data comes from

We used a hand-picked subset of **Aposemat IoT-23**, a public dataset released by the Stratosphere Lab at Czech Technical University. It's a collection of network captures recorded on real IoT devices that were either left in their normal state or deliberately infected with a known malware family. Each capture comes as a Zeek `conn.log`, which is basically one row per network connection with both a binary label (`Benign`/`Malicious`) and a more specific label for the malicious ones (`DDoS`, `C&C`, port-scan, etc.).

We took 12 of these captures — enough to cover several malware families without making the warehouse unmanageable on a 3-node student cluster.

2.2 How big is it

Property	**Value**
Total connections (rows)	25,011,247
Distinct captures	12
Per-row features (raw Zeek fields)	23
Target classes	7
Format on disk (cluster)	Parquet + Snappy
Approx. raw CSV size	∼737 MB

Table 2: Volume and shape of the working dataset.

The seven label values are: `Benign`, `Malicious`, `Malicious DDoS`, `Malicious PartOfAHorizontalPortScan`, `Malicious C&C`, `Malicious Attack`, `Malicious FileDownload`.

2.3 Architecture of the pipeline

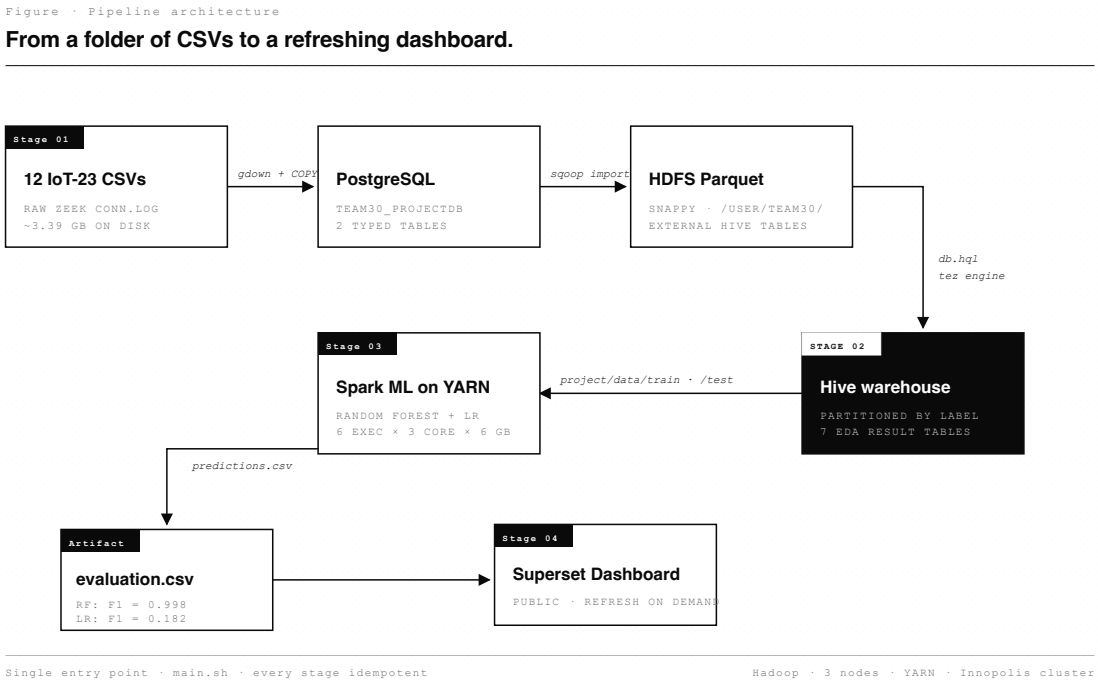

Figure 1: End-to-end architecture. CSVs go into Postgres; Sqoop lifts the typed tables onto HDFS as Parquet; Hive serves both the EDA queries and Spark; the trained model and the EDA result tables both feed a single Superset dashboard.

2.4 Per-stage input and output

Stage	Input	Output
Stage 1	12 IoT-23 capture CSV files	PostgreSQL tables `captures`, `network_connections`; HDFS Parquet/Snappy warehouse at `project/warehouse`
Stage 2	Sqoop Parquet warehouse on HDFS	Hive database `team30_projectdb` with bucketed/partitioned tables; 7 result tables `q1_results`...`q7_results`; CSV exports in `output/`
Stage 3	Hive table `network_connections_part` (label-partitioned)	Trained models `models/model1` (RF), `models/model2` (LR); predictions `output/modelN_predictions.csv`; metrics `output/evaluation.csv`
Stage 4	Hive result tables + `evaluation.csv`	Public Apache Superset dashboard

Table 3: Inputs and outputs of each pipeline stage.

3 Data Preparation

3.1 The relational model

In Postgres we have two tables and one foreign key. `captures` holds metadata about each of the 12 IoT-23 captures (one row per capture). `network_connections` holds every parsed Zeek connection (∼25M rows) and points back to its capture via `capture_id`. That's the whole schema — nothing fancy.

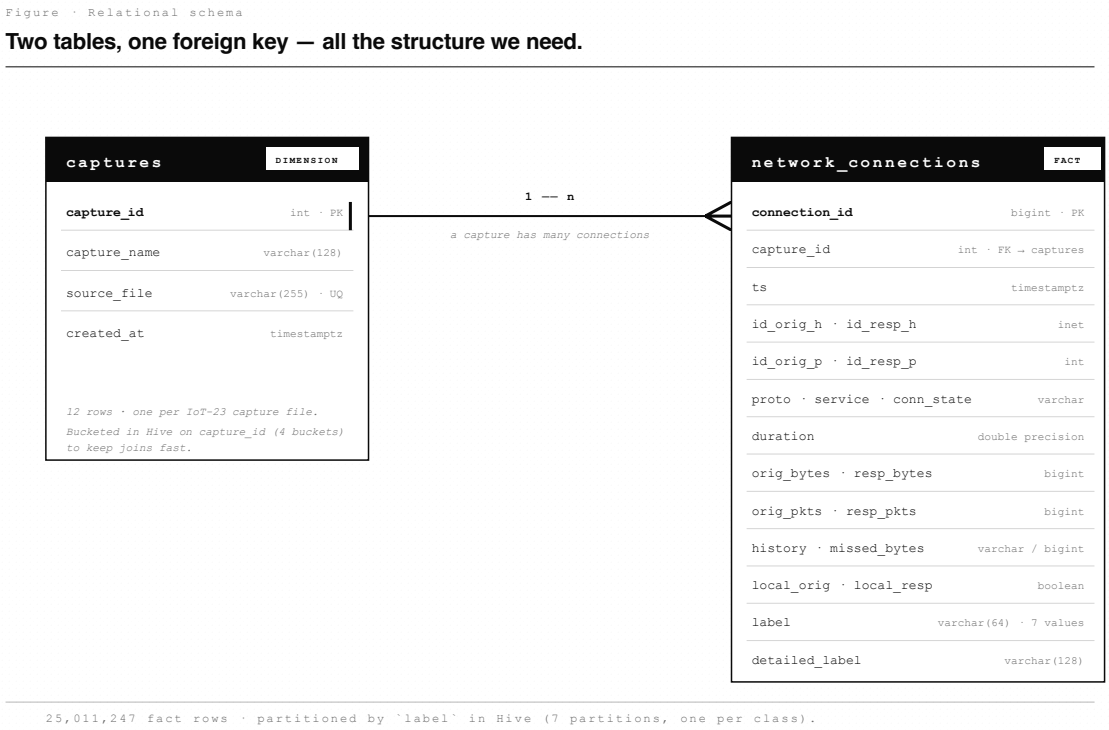

Figure 2: ER diagram. `captures` (1) — (N) `network_connections`. The crow's-foot on the right means *many connections per capture*.

To load the CSVs we go through a third table, `stg_network_connections`, which is just a staging area where every column is plain text. We `COPY` the CSV in as-is, then a transformation `INSERT` cleans the cells (regex out the trailing `.0` on integers, swap sentinel `-` for `NULL`, cast to `inet` and `timestamptz`) and writes the result into the typed `network_connections` table. Doing it in two steps means we never have to fight the Postgres CSV parser over IoT-23's quirks.

3.2 Sample records

A couple of rows from `network_connections` (truncated for width):

cap	ts	id_orig_h	proto	conn_state	orig_bytes	resp_bytes	orig_pkts	label
1	2018-12-21 14:02:11	192.168.1.132	tcp	S0	0	0	1	Malicious
9	2019-04-30 09:15:48	192.168.100.103	udp	SF	1024	64	4	Benign

Table 4: Two example records from the canonical `network_connections` table.

3.3 The Hive warehouse

The Hive database is called `team30_projectdb` and lives on top of the Sqoop output. We keep two physical layouts:

- `captures_bucketed` — 12 rows, but `CLUSTERED BY (capture_id) INTO 4 BUCKETS` so it joins cleanly against the fact table.
- `network_connections_part` — partitioned by `label`. Almost every query we ever run filters by label, so partitioning that way means Tez (and later Spark) can skip whole files instead of scanning everything.

The whole warehouse is rebuilt by `sql/db.hql`:

```
SET hive.execution.engine=tez;
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
SET hive.enforce.bucketing=true;

INSERT OVERWRITE TABLE network_connections_part PARTITION (label)
SELECT connection_id, capture_id, CAST(ts AS TIMESTAMP), uid,
       id_orig_h, id_orig_p, id_resp_h, id_resp_p,
       proto, service, duration, orig_bytes, resp_bytes,
       conn_state, local_orig, local_resp, missed_bytes, history,
       orig_pkts, orig_ip_bytes, resp_pkts, resp_ip_bytes,
       tunnel_parents, detailed_label, label
FROM network_connections_ext;
```

3.4 Picking a storage format

Before settling on Parquet + Snappy we ran a small benchmark: write the fact table four different ways, then time a full scan and one analytic query. Three runs each, results averaged:

Format	Codec	Runs	Avg write (s)	Avg full scan (s)	Avg analytic (s)
parquet	snappy	3	17.7	6.76	9.29
parquet	gzip	3	18.0	6.64	9.56
avro	snappy	3	17.0	9.32	10.50
avro	bzip2	3	19.0	8.36	11.01

Table 5: Storage format benchmark. Parquet beats Avro on scan time by a clear margin; Snappy and gzip are basically tied for Parquet, and Snappy costs less CPU. So we shipped Parquet + Snappy.

4 Data Analysis

The seven HiveQL queries (`sql/q1.hql`…`sql/q7.hql`) each materialise a small `qN_results` Parquet table and dump a CSV that the dashboard reads directly. The interpretations below are what we'd actually tell a teammate looking at the chart for the first time.

4.1 Insight 1 — How the labels split

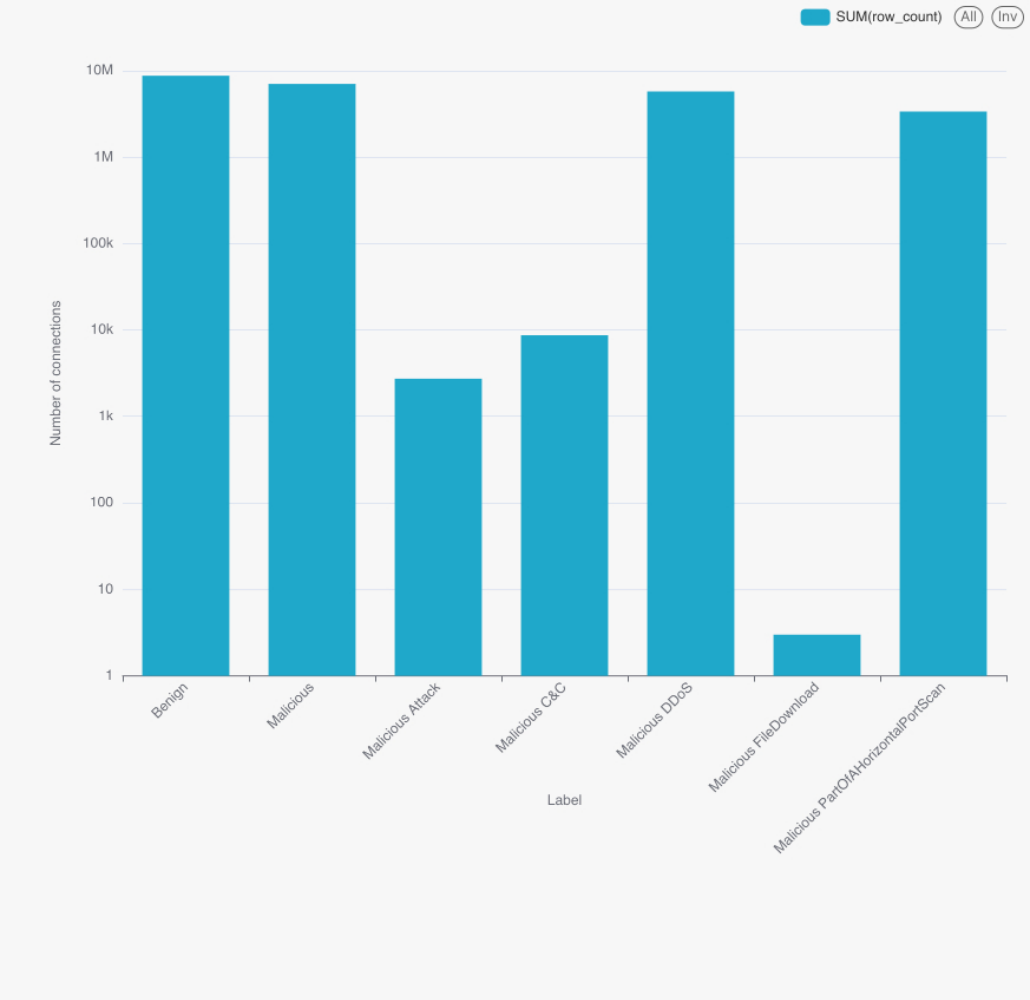

Figure 3: Class shares of `network_connections_part`.

Label	Rows	Share
Benign	8,780,158	35.11%
Malicious	7,055,007	28.21%
Malicious DDoS	5,778,154	23.10%
Malicious PartOfAHorizontalPortScan	3,386,241	13.54%
Malicious C&C	8,685	0.035%
Malicious Attack	2,755	0.011%
Malicious FileDownload	3	0.000%

Table 6: Label distribution.

The dataset isn't balanced — four classes hold 99.9% of the rows — but it's not flat either. The three rare classes (C&C, Attack, FileDownload) are interesting precisely *because* they're rare: collapsing them into a single "malicious" bucket would lose a real signal. That's why we kept the task multiclass instead of going binary.

4.2 Insight 2 — Which protocol gets used

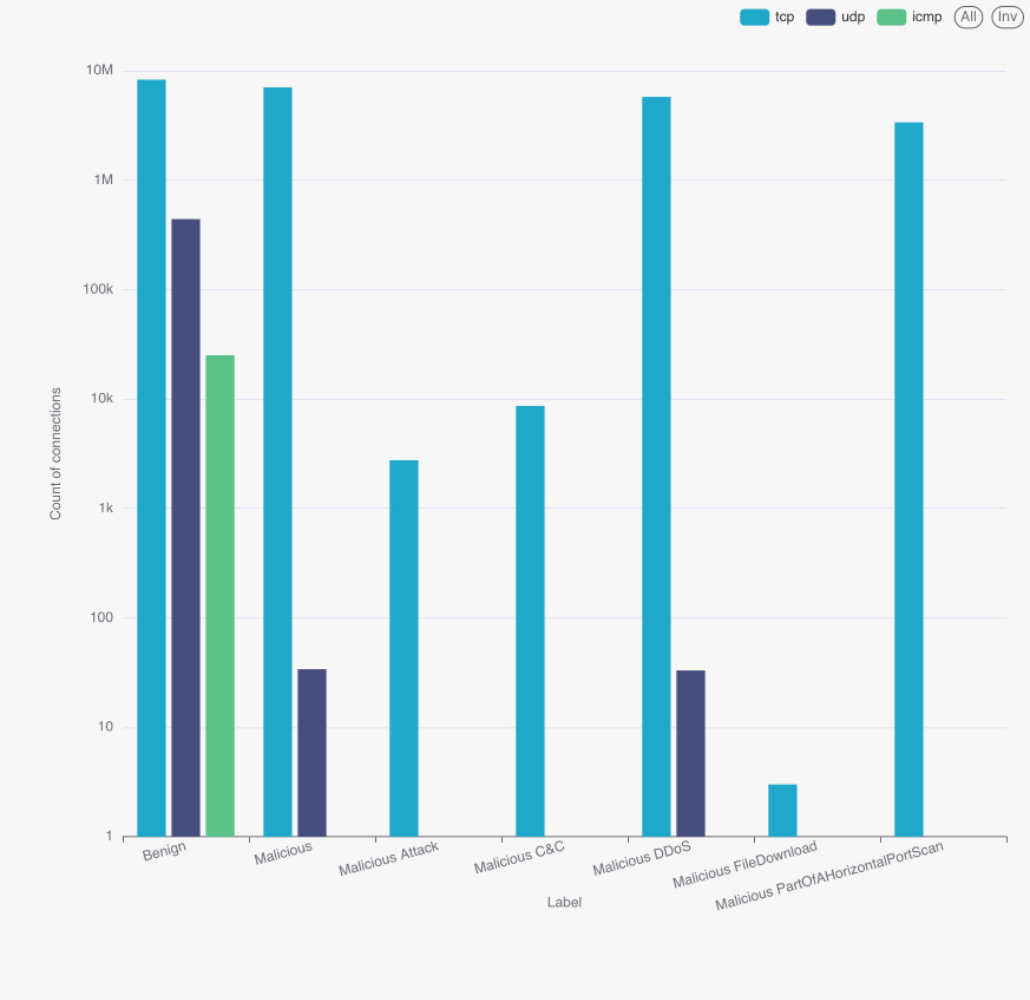

Figure 4: Protocol usage per label. UDP shows up almost exclusively in benign DNS-style traffic; almost every malicious connection is TCP.

If you see a UDP flow on this network, it's almost certainly fine — DNS, NTP, the usual. If you see a malicious flow, it's basically always TCP. So `proto` on its own won't separate one malicious class from another, but it's a strong prior against "UDP = bad".

4.3 Insight 3 — How big the connections are

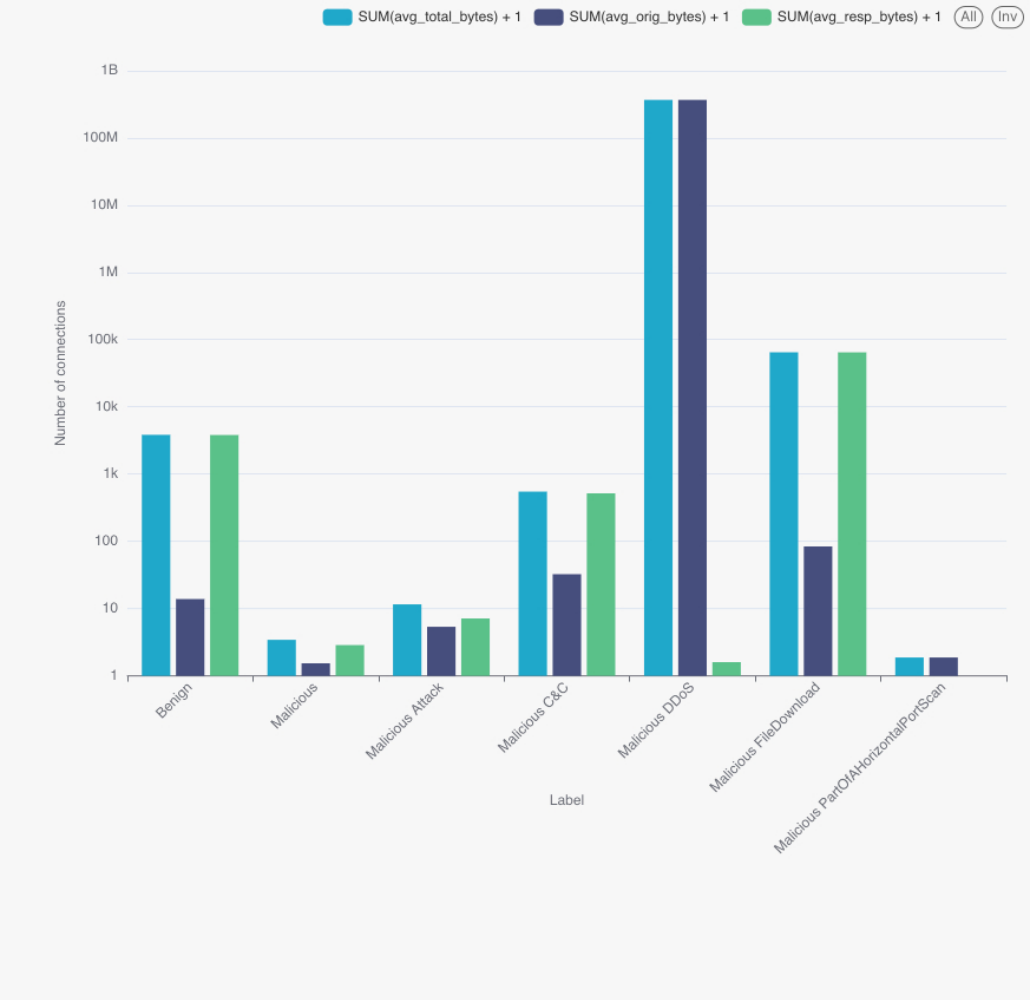

Figure 5: Average originator and responder bytes per connection by label.

Label	Avg orig bytes	Avg resp bytes	Avg total bytes
Benign	12.87	3,830.63	3,843.51
Malicious	0.54	1.87	2.41
Malicious DDoS	3.71×10^8	0.60	3.71×10^8
Malicious C&C	31.48	517.93	549.41
Malicious Attack	4.35	6.16	10.51
Malicious PartOfAHorizontalPortScan	0.87	0.00	0.87
Malicious FileDownload	83.33	64,982.00	65,065.33

Table 7: Average per-connection byte counts by label.

DDoS jumps out immediately — 3.7×10^8 originator bytes per flow on average, basically six orders of magnitude over benign. That's the amplification payload. At the other end, port scans and the generic `Malicious` class barely move a byte per connection: they're basically just SYN probes. C&C looks more like a normal long-lived session (a few KB on the responder side), and FileDownload (only 3 rows) shows the asymmetric pattern you'd expect.

4.4 Insight 4 — Each capture is one behaviour

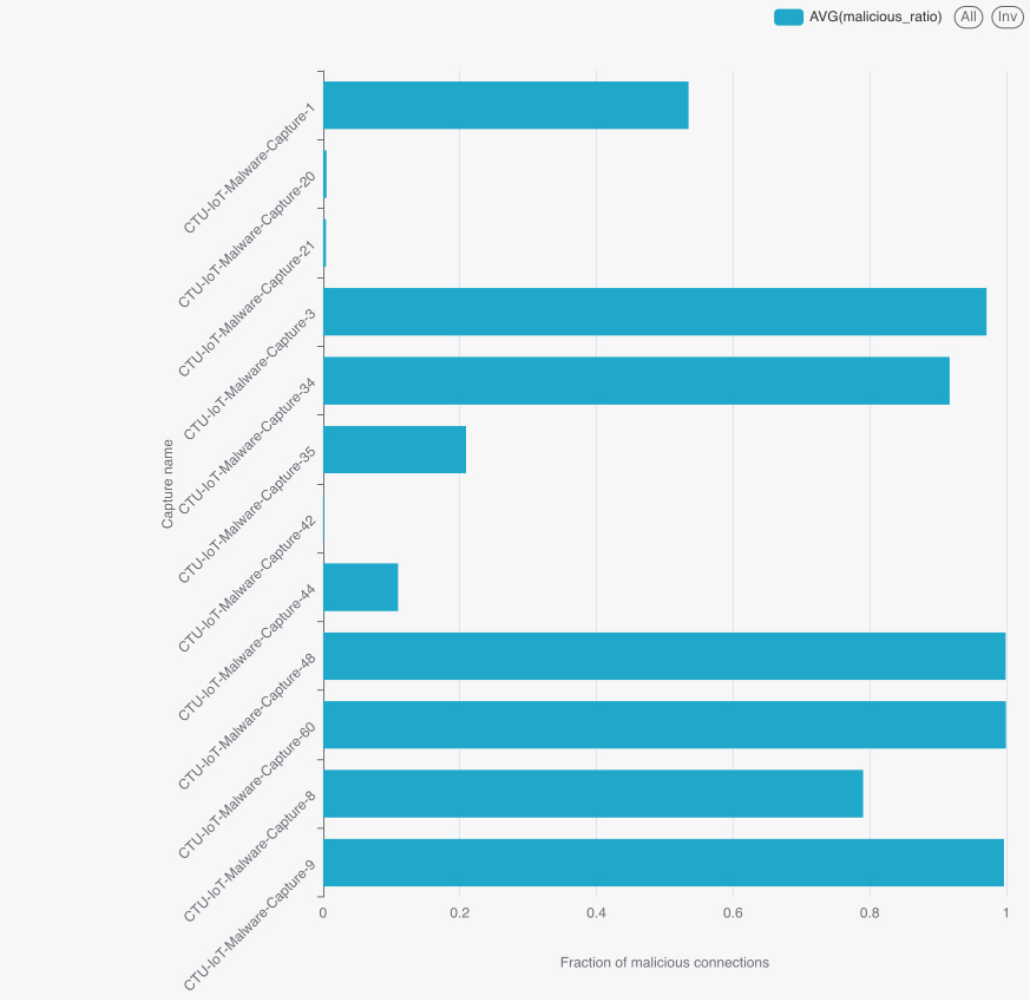

Figure 6: Total flows and malicious ratio per capture.

The captures don't sit anywhere in the middle: 8 of them are over 79% malicious, 4 of them are under 11%. There's no "slightly suspicious" capture. That makes sense — each capture was recorded around a single infection, so basically the whole capture is either compromised traffic or a clean baseline.

The practical consequence: if we ever do a real evaluation, we should split by capture, not by row. Random row-level splitting (which is what we used for this report) leaks the same capture into both train and test. We flagged it in the conclusion.

4.5 Insight 5 — What the connection state looks like

Almost everything in this dataset is `S0` — a SYN that never gets answered. Even the benign traffic is dominated by IoT devices repeatedly trying to reach a server that's not there. DDoS is the exception: it splits between `OTH` (we joined the flow mid-stream) and `RSTOS0` (the originator gets a RST after the SYN). C&C shows the classic `S3` fingerprint — the originator finishes but the responder is still talking, which is what a long-running control channel looks like.

Label	Dominant `conn_state`	Rows	Share within label
Benign	S0	8,722,217	99.3%
Malicious	S0	7,036,199	99.7%
Malicious DDoS	OTH + RSTOS0	5,777,712	99.99%
Malicious C&C	S0 + S3	7,655	88%
Malicious PartOfAHorizontalPortScan	S0	3,386,241	100%

Table 8: Dominant TCP connection state per label.

4.6 Insight 6 — Where the attacks are aimed

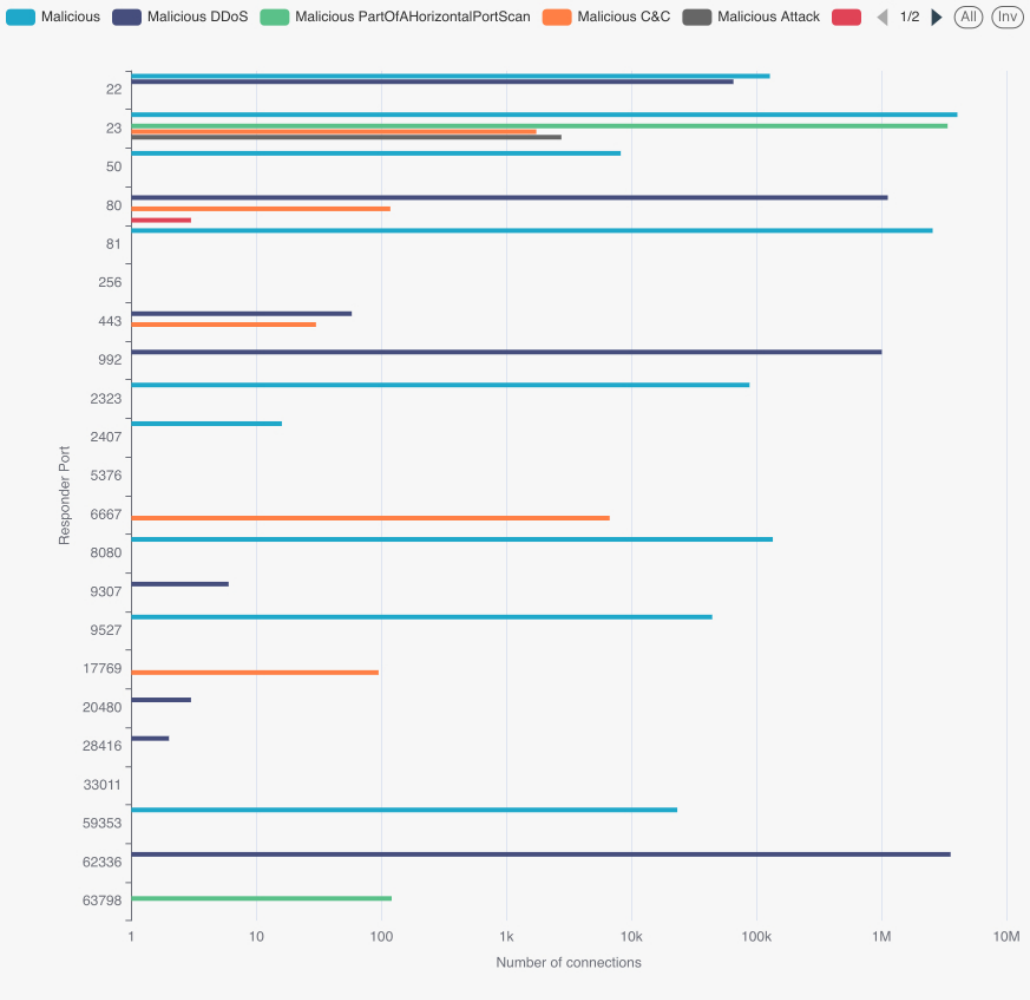

Figure 7: Top targeted destination ports per non-Benign label.

Telnet (port 23) is, by a huge margin, the most attacked destination — 4M generic Malicious flows and another 3.4M port-scan flows hit it. That's the Mirai pattern: scan the internet for IoT devices with default Telnet credentials, log in, install yourself, repeat. DDoS prefers the high port `62336` (a Mirai control/echo port) and HTTP `80`. C&C mostly uses IRC port `6667`, which is also a Mirai-family classic.

Label	Port	Flows
Malicious	23	4,055,327
Malicious DDoS	62336	3,578,391
Malicious PartOfAHorizontalPortScan	23	3,386,119
Malicious	81	2,571,979
Malicious DDoS	80	1,125,806
Malicious DDoS	992	1,008,351
Malicious C&C	6667	6,706

Table 9: Top 7 targeted destination ports across non-Benign labels.

4.7 Insight 7 — How many packets per connection

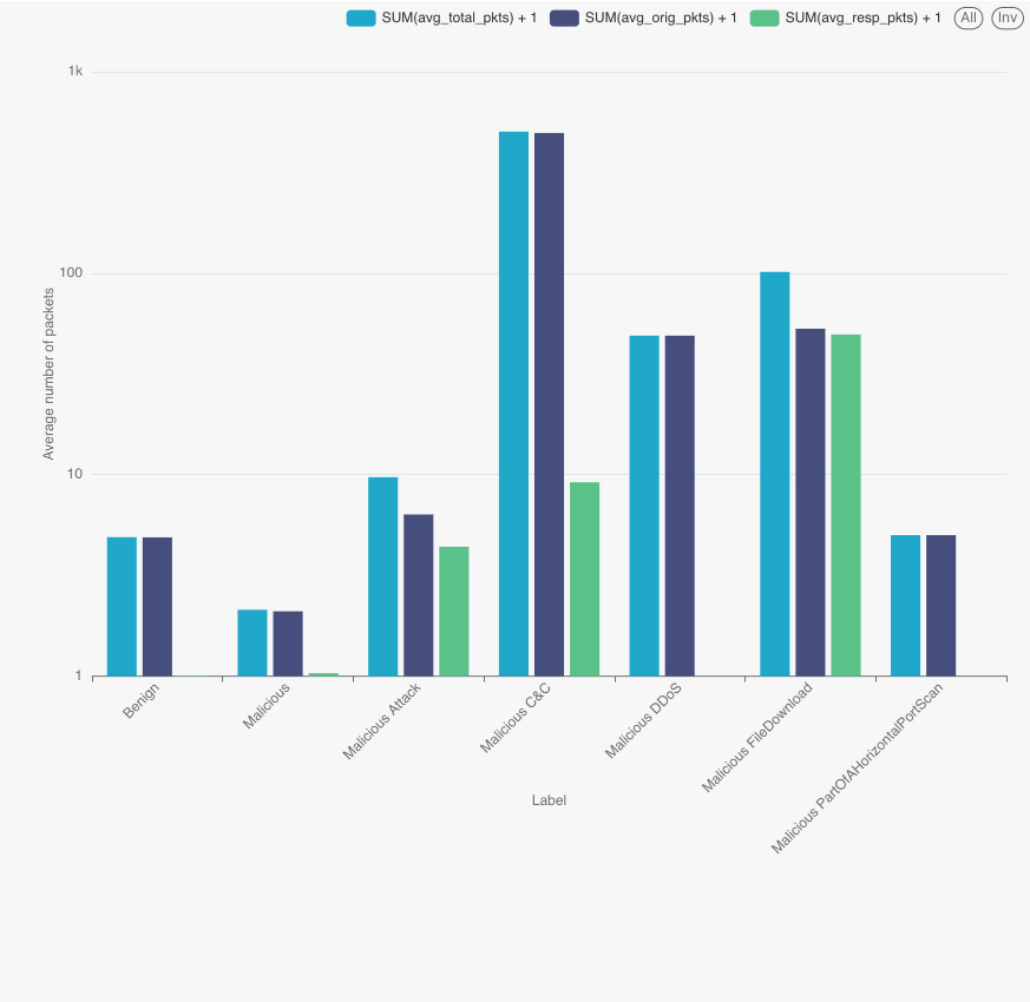

Figure 8: Average originator and responder packets per connection by label.

The interesting row here is C&C: about 495 packets out per session, with a real (8-packet) response. That's a genuine long-running conversation, very different from the one-shot SYN of port scans (where the responder is essentially zero). The combination of `orig_pkts`, `resp_pkts` and `conn_state` ends up doing most of the work for the Random Forest later.

5 ML Modeling

5.1 Building the feature pipeline

`scripts/ml_data_preparation.py` fits a single PySpark `Pipeline` on the training fold (so the encoders never see the test rows). Five steps:

1. **Cyclical time encoding.** A small custom transformer pulls year, month, day, hour, minute, second out of `ts` and adds sin/cos for the periodic ones. Without this the model would learn that `hour=23` and `hour=0` are far apart.
2. **Imputation.** Drop `tunnel_parents` (always null); fill string nulls with `"unknown"`, numeric nulls with 0, booleans with `False`.
3. **Categorical encoding.** `StringIndexer + OneHotEncoder` for `proto`, `service`, `conn_state`, `history`.
4. **Feature selection.** A `ChiSqSelector(fpr=0.1)` on the categorical block keeps only the dummies that actually correlate with the label.
5. **Assemble + scale.** `VectorAssembler` concatenates everything; `StandardScaler(withMean=False, withStd=True)` gives us the final `features` column.

The label gets its own `StringIndexer`. We split 60/40 with seed 42, and write the resulting train/test out to HDFS as Parquet so training and evaluation read from the same on-disk artefact.

5.2 Key optimisations that made it fit on the cluster

Three things turned a slow, fragile training loop into something we could re-run reliably inside our YARN budget. They are not glamorous, but they are the reason the ML stage finished at all on a contended cluster.

- **Persisting the training set in memory + disk.** A `CrossValidator` re-reads the training fold once per hyperparameter combination and once per fold. Without caching, every one of those reads goes back to HDFS, decodes Parquet, re-runs the feature pipeline, and shuffles — which is most of the wall-clock time. We call `train_data.persist(StorageLevel.MEMORY_AND_DISK)` after the initial read so the second and subsequent passes hit RAM (or local disk if a partition spills). This alone cut the per-fit time roughly in half on our profile and let the CV runs survive single-executor losses without falling back to HDFS.
- **Selecting categorical features with `ChiSqSelector`.** One-hot encoding `proto`, `service`, `conn_state` and `history` produces several hundred sparse dummy columns, most of which carry no signal for the label. Feeding all of them into Random Forest blows up split-search time, and into Logistic Regression blows up the optimiser's inner loop. We run `ChiSqSelector(fpr=0.1)` over the categorical block, fitted on the training fold only, and keep only the dummies whose chi-squared p-value clears the threshold. The feature vector shrinks by an order of magnitude and the model fits faster without measurable loss on the held-out F1.
- **Writing train/test as Parquet with explicit sharding.** Once the feature pipeline is fitted, we materialise the transformed train/test sets back to HDFS:

```
train_data.select("features", "label") \
    .coalesce(4) \
    .write.mode("overwrite") \
    .format("parquet") \
    .save("project/data/train")
```

Two things matter here. First, we write Parquet so the training script can re-read the already-vectorised features without re-running the feature pipeline (it would otherwise do that on every `spark-submit`). Second, the explicit `coalesce(4)` controls the number of output files: too many and the executor open-file overhead dominates; too few and we lose parallelism on the read. Four shards matched our executor layout (6 executors × 3 cores) reasonably well for both training and evaluation. Combined with persistence above, the next `ml_models_training.py` run starts from a vectorised, sharded Parquet file and goes straight into the cross-validator.

5.3 Two models, one cross-validator

We trained two models, both wrapped in `CrossValidator(numFolds=2, parallelism=3)` optimising multiclass `f1`. Each `spark-submit` ran on YARN with 6 executors × 3 cores × 6 GB. The training set is `persist(MEMORY_AND_DISK)`'d so the CV folds don't repeat the read.

Model	Grid	Best
Random Forest	numTrees $\in \{30, 60\}$ maxDepth $\in \{4, 6\}$ impurity = gini (default)	numTrees = 60 maxDepth = 6 impurity = gini
Logistic Regression (multinomial)	regParam $\in \{0.01, 0.1\}$ elasticNetParam $\in \{0.0, 0.5\}$ maxIter = 50, family = multinomial	regParam = 0.01 elasticNet = 0.0

Table 10: Hyperparameter grid and the values picked by 2-fold cross-validation on the multiclass `f1` metric.

5.4 How they did

The evaluation script (`scripts/ml_evaluation.py`) reads the HDFS-persisted predictions and computes four metrics with PySpark's `MulticlassClassificationEvaluator`:

Model	Accuracy	Precision	Recall	F1
Random Forest	**0.9979**	**0.9979**	**0.9979**	**0.9978**
Logistic Regression	0.3510	0.3395	0.3510	0.1824

Table 11: Held-out evaluation on ∼10M test rows.

The Random Forest hits $F_1 = 0.998$ across all seven classes — about 5.5× better than the linear baseline on the same features. That gap isn't surprising: the decision boundary on `conn_state` + `id_resp_p` + `orig_pkts` is sharply non-linear after one-hot encoding, and a multinomial linear model just can't represent it. We could spend more time tuning LR, but that would be polishing a wrong tool.

We persist the predictions to HDFS as CSV before evaluating, which means we can re-run the evaluator without re-running the trainer — useful if we want to add a new metric later.

6 Data Presentation

6.1 The dashboard

The defence-day dashboard lives in **Apache Superset** and connects to the cluster's HiveServer2 endpoint at `jdbc:hive2://hadoop-03.uni.innopolis.ru:10001`. It's one public dashboard, refreshed on demand against the same result tables that Stages 2 and 3 produced. Six sections:

A. Data characteristics. Headline numbers: 25M rows, 23 features, 7 classes, 12 captures. The first thing a viewer sees.

B. Class balance (Insight 1). Donut and percentage list, sourced from `q1_results`.

C. Behaviour signatures (Insights 2, 3, 5, 7). Stacked-bar protocol mix, average bytes/-packets per label, dominant `conn_state` per label.

D. Top targets (Insight 6). Port histograms per non-Benign label, with a callout on port 23.

E. Capture strata (Insight 4). Per-capture maliciousness ratio bar chart — the bimodal capture-level pattern is hard to miss.

F. Model performance. Side-by-side comparison of the two models, sourced from `evaluation.csv`.

6.2 What the dashboard tells you

Three things stand out once everything is in front of you:

1. **The class skew is interesting, not degenerate.** Four classes carry the bulk of the data and three rare classes carry real diagnostic signal. Treating the task as multiclass keeps that signal visible.
2. **The attack surface is concentrated.** Port 23 alone accounts for more than 7M malicious flows. Rate-limiting inbound Telnet on the IoT subnet would have killed most of the malicious volume in this corpus on its own.
3. **The right model wins by a lot.** With four hyperparameter combinations and a 2-fold CV, the Random Forest reaches $F_1 = 0.998$. The same problem is essentially unsolvable for a multinomial linear model. So a tree ensemble is the right starting point for any production version of this.

7 Conclusion

7.1 What we shipped

A reproducible Hadoop pipeline that takes 25M Zeek connection records, loads them into a Hive warehouse on HDFS, runs seven HiveQL queries on Tez to surface the structure of the data, trains two Spark ML models with cross-validation on YARN, and exposes everything in a public Apache Superset dashboard. Random Forest reaches $F_1 = 0.998$ on a held-out test set of about 10M rows. Every artefact in `output/` can be regenerated by running `main.sh`.

7.2 How we worked

The role split worked well. Ivan owned ingestion and the Hive layout; Kirill owned the Spark feature pipeline and the model grid; Nikita owned EDA and the dashboard story; Dmitry owned documentation and ran the re-run checks. The "one `main.sh`, every stage idempotent" rule was probably the single most useful constraint — it forced us to clean up before doing anything, instead of relying on whatever was left from the last run.

7.3 What gave us trouble

- **Loading the CSVs.** The raw Zeek files use `-` for missing values and write integers as floats (`1234.0`). The Postgres transformation insert had to handle both with `NULLIF` + `REGEXP_REPLACE` before casting. One missed regex meant a full reload.

- **Partition skew in Hive.** Partitioning by `label` is great for queries but makes one tiny partition (FileDownload, 3 rows) and four huge ones. Fine for analytical scans, painful for Spark shuffles — we worked around it with `repartition(18)` after the read.
- **Cluster budget vs. ML grid size.** We started with the plan from the project brief: three model families, three hyperparameters per model with three values each ($3^3 = 27$ combinations per model), and a 3-fold cross-validation. On paper that's $3 \times 27 \times 3 = 243$ training runs. On the shared Innopolis cluster, where the YARN queue is contended with several other teams and our 25M-row training set takes 8–12 minutes for a single fit on the requested executor profile ($6 \times 3 \times 6$ GB), that grid would have meant 30–50 hours of wall-clock time — and we kept getting pre-empted or queue-throttled in the middle of CV runs. After two failed full grids we cut the scope deliberately: two model families (RF, multinomial LR), a 2×2 grid per model, and $k = 2$ CV. It's not what the brief asked for, but it's what fit in the budget while still letting both models finish a clean held-out evaluation. We've kept the original code path intact so the larger grid can be re-enabled the moment the cluster has spare capacity.
- **Logistic Regression went sideways.** The multinomial LR barely beat majority-class on this feature set. We chose to report it honestly rather than spend more cycles tuning a model that was structurally the wrong fit.
- **Pushing from the cluster.** The cluster's DNS doesn't resolve `github.com`, so pushing artefacts up needed a fixed `HostName` entry in `~/.ssh/config`.

7.4 What we'd do next

1. Split by `capture_id` instead of by row. Insight 4 basically tells us we have to.
2. Add Gradient-Boosted Trees to the grid. They should beat the Random Forest on a feature space that's mostly one-hot dummies.
3. Wire the trained Random Forest into Spark Structured Streaming over the same Hive partitions, so it can score new traffic in near real time.
4. Add per-class precision/recall and a confusion-matrix heatmap to the dashboard. Right now the model panel only shows aggregate metrics, which hides where the rare classes go wrong.

7.5 Who did what

Task	Ivan I.	Kirill S.	Nikita Z.	Dmitry T.	Avg hrs	Deliverables
Dataset selection & access	25%	25%	25%	25%	6	IoT-23 captures
Stage 1 — PostgreSQL schema & load	60%	10%	10%	20%	20	`create_tables.sql`, `build_projectdb.py`
Stage 1 — Sqoop & HDFS warehouse	70%	10%	0%	20%	16	`import_to_hdfs.sh`
Stage 1 — Storage benchmark	40%	40%	0%	20%	12	`benchmark_storage_*`, CSVs
Stage 2 — Hive warehouse setup	40%	20%	20%	20%	18	`db.hql`, partitioned tables
Stage 2 — 7 EDA queries	10%	0%	80%	10%	24	`q1.hql` ... `q7.hql`
Stage 2 — EDA charts & storytelling	10%	0%	80%	10%	16	`output/q*.jpg`, `q*.csv`
Stage 3 — Feature pipeline	10%	75%	10%	5%	28	`ml_data_preparation.py`
Stage 3 — Model training & CV	10%	80%	5%	5%	32	`ml_models_training.py`, `models/`
Stage 3 — Evaluation	10%	60%	20%	10%	14	`ml_evaluation.py`, `evaluation.csv`
Stage 4 — Superset dashboard	10%	10%	70%	10%	20	Public dashboard
Pipeline orchestration (`main.sh`)	50%	20%	10%	20%	16	`main.sh`, `stage*.sh`
Repository documentation	10%	10%	10%	70%	14	`README.MD`, `docs/`
Final report (this document)	5%	5%	10%	80%	40	`report/report.pdf`
Defence presentation	10%	20%	30%	40%	24	`presentation/presentation.pdf`
QA & reproducibility checks	15%	15%	15%	55%	20	Pipeline reruns
Total person-hours					**320**	

Table 12: Contribution table. Per-row percentages sum to 100% across the four team members; the *Avg hrs* column is the team-wide person-hours spent on each task.

Repository: github.com/Dmitry057/big-data-group-project
Local path on cluster: `/home/team30/project30` (user `team30`)